

Artificial Intelligence – Assignment 3

Solving the 8-Puzzle Problem Using A* Search

1. Problem Description

The 8-puzzle is a classic AI search problem consisting of a 3x3 board with eight numbered tiles (1-8) and one blank space. Starting from a given (scrambled) arrangement of tiles, the goal is to reach a specified goal arrangement by sliding tiles into the blank space, one move at a time. Only the tile adjacent to the blank (up, down, left, or right) can move into it.

The problem is a well-known benchmark for studying state-space search and heuristic search algorithms in Artificial Intelligence, because its state space ($9!/2 = 181,440$ reachable states) is small enough to solve quickly, yet large enough to meaningfully demonstrate the benefit of a good heuristic over blind search.

Goal State used in this implementation:

1	2	3
4	5	6
7	8	_

Not every scrambled arrangement can reach the goal state - exactly half of all possible arrangements are solvable. This is determined using the concept of “inversions”: an inversion is any pair of tiles that appear in the wrong relative order compared to the goal. A configuration is solvable if and only if its number of inversions is even.

2. Program Implementation

The program is implemented in Python 3 and solves the 8-puzzle using the A* search algorithm with the Manhattan Distance heuristic. A* was chosen because it guarantees the shortest solution path (optimality) while expanding far fewer nodes than uninformed methods such as BFS, since it uses the heuristic to guide the search toward the goal.

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def get_neighbors(state):
    idx = state.index(0)
    r, c = divmod(idx, 3)
    moves = []
    dirs = {'Up': (-1, 0), 'Down': (1, 0), 'Left': (0, -1), 'Right': (0, 1)}
    for name, (dr, dc) in dirs.items():
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            nidx = nr * 3 + nc
            new_state = list(state)
            new_state[idx], new_state[nidx] = new_state[nidx], new_state[idx]
            moves.append((name, tuple(new_state)))
    return moves

def manhattan(state):
    dist = 0
    for i, val in enumerate(state):
        if val == 0:
            continue
        goal_idx = val - 1
        r1, c1 = divmod(i, 3)
        r2, c2 = divmod(goal_idx, 3)
        dist += abs(r1 - r2) + abs(c1 - c2)
    return dist

def is_solvable(state):
    arr = [x for x in state if x != 0]
    inversions = 0
```

```

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] > arr[j]:
            inversions += 1
return inversions % 2 == 0

def a_star(start):
    if not is_solvable(start):
        return None, 0
    counter = 0
    g_score = {start: 0}
    f_score = manhattan(start)
    open_set = [(f_score, counter, start, [])]
    closed = set()
    nodes_expanded = 0

    while open_set:
        f, _, current, path = heapq.heappop(open_set)
        if current in closed:
            continue
        closed.add(current)
        nodes_expanded += 1

        if current == GOAL:
            return path, nodes_expanded

        for move_name, neighbor in get_neighbors(current):
            tentative_g = g_score[current] + 1
            if neighbor in closed and tentative_g >= g_score.get(neighbor, float('inf')):
                continue
            if tentative_g < g_score.get(neighbor, float('inf')):
                g_score[neighbor] = tentative_g
                counter += 1
                new_f = tentative_g + manhattan(neighbor)
                heapq.heappush(open_set, (new_f, counter, neighbor, path + [(move_name,
neighbor)]))

    return None, nodes_expanded

def print_state(state):
    for i in range(0, 9, 3):
        row = state[i:i + 3]
        print(' '.join(str(x) if x != 0 else '_' for x in row))
    print()

def run_test(name, start):
    print(f"===== {name} =====")
    print("Initial State:")
    print_state(start)
    path, expanded = a_star(start)
    if path is None and not is_solvable(start):
        print("Result: UNSOLVABLE puzzle configuration.")
        return
    print(f"Solved in {len(path)} moves. Nodes expanded: {expanded}")
    print("Move sequence:", ' -> '.join(m for m, _ in path))

if __name__ == "__main__":
    test1 = (1, 2, 3, 4, 0, 6, 7, 5, 8)
    test2 = (1, 2, 3, 4, 5, 6, 0, 7, 8)
    test3 = (1, 2, 3, 5, 0, 6, 4, 7, 8)
    test4 = (1, 2, 3, 4, 5, 6, 8, 7, 0) # unsolvable

    run_test("Test Case 1 (Easy, 2 moves)", test1)
    run_test("Test Case 2 (Simple, few moves)", test2)

```

```
run_test("Test Case 3 (Moderate difficulty)", test3)
run_test("Test Case 4 (Unsolvable configuration)", test4)
```

3. Input and Output

The program takes the initial board configuration as input (hard-coded as test cases below, though it can easily be modified to accept user input). For each test case, it prints the initial state, whether the puzzle is solvable, the sequence of moves found, the total number of moves, and the number of nodes expanded during the search. Below is the actual console output produced when the program was run:

```
>>> python3 eight_puzzle.py
===== Test Case 1 (Easy, 2 moves) =====
Initial State:
1 2 3
4 _ 6
7 5 8

Solved in 2 moves. Nodes expanded: 3
Move sequence: Down -> Right

===== Test Case 2 (Simple, few moves) =====
Initial State:
1 2 3
4 5 6
_ 7 8

Solved in 2 moves. Nodes expanded: 3
Move sequence: Right -> Right

===== Test Case 3 (Moderate difficulty) =====
Initial State:
1 2 3
5 _ 6
4 7 8

Solved in 4 moves. Nodes expanded: 5
Move sequence: Left -> Down -> Right -> Right

===== Test Case 4 (Unsolvable configuration) =====
Initial State:
1 2 3
4 5 6
8 7 _

Result: UNSOLVABLE puzzle configuration.
```

4. Explanation of the Code

- **GOAL**: a tuple representing the solved board, read left-to-right, top-to-bottom, with 0 representing the blank tile.
- **get_neighbors(state)**: locates the blank tile's row/column and generates all valid states reachable by sliding an adjacent tile Up, Down, Left, or Right into the blank.
- **manhattan(state)**: the heuristic function $h(n)$. For every tile, it computes the sum of the horizontal and vertical distances between the tile's current position and its position in the goal state. This heuristic is admissible (never overestimates) and is the standard choice for the 8-puzzle.
- **is_solvable(state)**: counts inversions (pairs of tiles out of relative order) in the tile sequence, ignoring the blank. An even inversion count means the puzzle can be solved.
- **a_star(start)**: implements A* search using Python's `heapq` as a priority queue. Each entry stores $f(n) = g(n) + h(n)$, where $g(n)$ is the number of moves taken so far and $h(n)$ is the Manhattan distance. The algorithm repeatedly expands the state with the lowest $f(n)$, stopping when the goal is reached; a closed set prevents re-expanding already-processed states.

- `print_state(state) / run_test(name, start)`: helper functions used to display the board and run the algorithm on a given test case, printing the move sequence and search statistics.

5. Test Cases

A minimum of three solvable configurations, plus one unsolvable configuration to test the solvability check, were used:

Test Case	Initial State	Goal Reached?	Moves	Nodes Expanded
1 - Easy	1 2 3 / 4 _ 6 / 7 5 8	Yes	2 (Down, Right)	3
2 - Simple	1 2 3 / 4 5 6 / _ 7 8	Yes	2 (Right, Right)	3
3 - Moderate	1 2 3 / 5 _ 6 / 4 7 8	Yes	4 (Left, Down, Right, Right)	5
4 - Unsolvable	1 2 3 / 4 5 6 / 8 7 _	No (odd inversions)	-	-

6. Applications in Real Life

Although the 8-puzzle itself is a toy problem, the search techniques it demonstrates are widely used:

- Robot motion and path planning, where a robot must find an optimal sequence of moves from a start configuration to a goal configuration.
- GPS and route-finding systems, which use A* (or variants of it) to find the shortest/fastest route using heuristics such as straight-line distance.
- Puzzle and game AI, including sliding-tile games, Rubik's Cube solvers, and other combinatorial puzzle solvers.
- Automated planning and scheduling systems, where a sequence of actions must be found to transform an initial state into a goal state at minimum cost.
- Network packet routing, where nodes represent routers and the goal is the lowest-cost path to a destination.

7. Challenges Faced

- Ensuring the heuristic (Manhattan distance) remains admissible so that A* still guarantees the optimal (shortest) solution.
- Detecting unsolvable configurations in advance (via the inversion-count rule) instead of letting the search run forever on an impossible puzzle.
- Avoiding re-expansion of already-visited states, which required careful bookkeeping with a closed set and updated g-scores.
- Managing tie-breaking in the priority queue when multiple states share the same $f(n)$ value, handled here with an incrementing counter alongside each queue entry.

8. Conclusion

This assignment implemented and evaluated the 8-puzzle problem using the A* search algorithm with the Manhattan distance heuristic. The program correctly distinguishes solvable from unsolvable configurations, finds the optimal (minimum-move) solution for solvable puzzles, and does so while expanding very few nodes compared to uninformed search strategies. The exercise reinforced core AI concepts including state-space representation, heuristic design, and informed search, all of which generalize to more complex real-world planning and pathfinding problems.

9. References

1. Russell, S., & Norvig, P. – Artificial Intelligence: A Modern Approach (Pearson).
2. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." IEEE Transactions on Systems Science and Cybernetics.

3. Python Software Foundation – heapq module documentation, [docs.python.org](https://docs.python.org/3/library/heapq.html).
4. GeeksforGeeks – “8 puzzle Problem using Branch and Bound / A* Search”, [geeksforgeeks.org](https://www.geeksforgeeks.org/8-puzzle-problem-using-branch-and-bound/).